



— CLIENT BRIEF

# Digital healthcare platform

I want people to book a doctor the way they book a table. Pick the specialist,  
see the real free times, take one, and know it is theirs.

— YOUR CLIENT

# What I am trying to do

I am putting a network of clinics online. Family doctors, cardiologists, dermatologists, paediatricians, spread over several clinics in a few cities.

Today a patient who wants an appointment calls the reception desk. Somebody with a paper diary answers, reads out two times that may or may not still be free, writes a name in a box, and that is the booking. If the patient wants to move it, they call again. If the doctor is ill, somebody calls twenty people.

Two things go wrong constantly. The same slot gets promised to two people, because two receptionists were writing at the same time. And nobody can tell a patient what is actually free without picking up a phone.

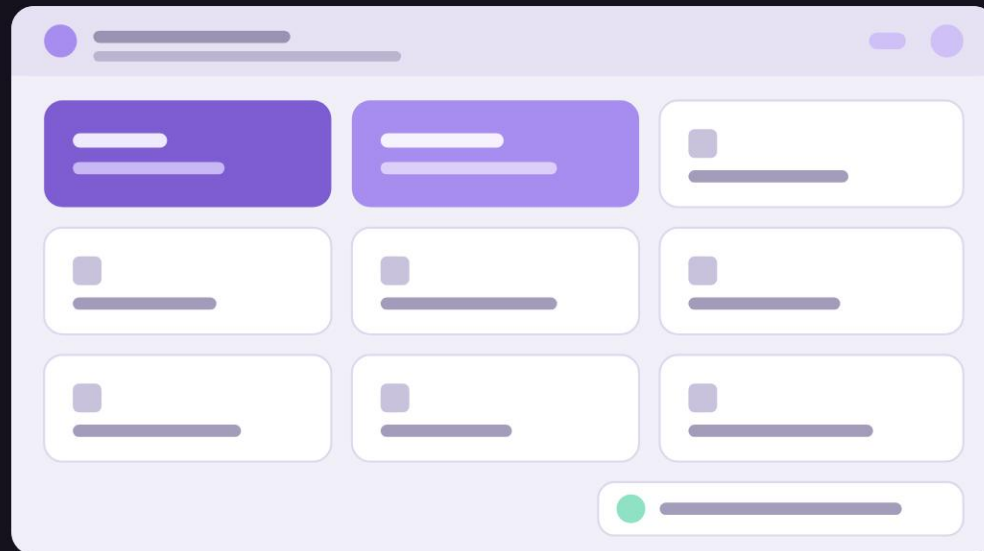
So I want a website where the patient does it themselves: find the doctor, see the real calendar, take a free time, and get it confirmed on the spot. And where the doctor opens their own screen in the morning and sees who is coming.

I am not asking you to build a hospital system. I am asking you to make booking a doctor honest: what the screen shows is free is genuinely free, and what is taken can never be taken twice.

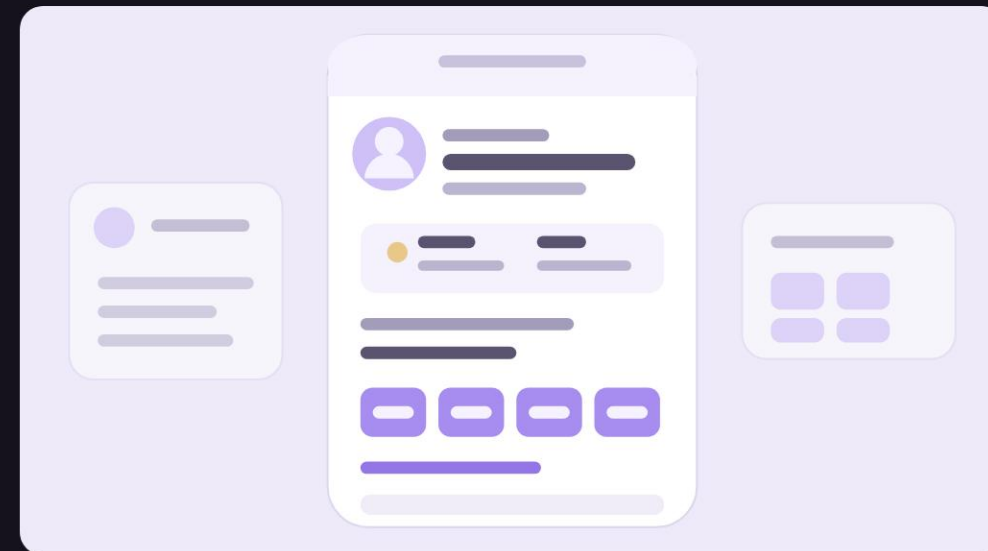
— REFERENCE

# The product I keep pointing at

Helsi. Ukraine's booking app for doctors: you search a specialist, open their card, and take a time from a row of buttons. I want that, smaller, and mine.



CLINIC SIDE, EVERYTHING ONE TILE AWAY



PATIENT SIDE, A DOCTOR AND FOUR FREE TIMES

*Take from it the doctor card, the row of time buttons and the calm density. Do not take the rest of it: no medical records, no prescriptions, no state registries, no pharmacy. Those are their ten years, not our six weeks.*

— YOUR CLIENT

# What I want to be able to do

## AS A PATIENT

- Register once and log in, without a receptionist involved
- Search a specialty, a doctor's name or a clinic
- Narrow it by city or clinic, by format, and by the day I am free
- Open a doctor and see who they are, where they work and what they charge
- See their real calendar and take a free time in two clicks
- Move my appointment to another time without calling anyone
- Cancel it, and have that time go back to other people
- See what is coming and what already happened

## AS A DOCTOR

- Log in and see today and this week, in order, by time
- See who is coming, for what, and in which format
- See only my own patients and my own calendar, never a colleague's
- Have my working hours be what generates the times patients can take
- Mark an appointment as completed, or as a patient who did not arrive

*The doctor side is small on purpose. It is not a second product. It is the proof that the same appointment looks right from both ends.*

— THE PRODUCT IN ONE LINE

# In simple words



01

Find a doctor



02

Read their profile



03

See real free times



04

Take one, get it  
confirmed



05

Move it or cancel it



06

The doctor sees you  
coming

*Six steps. The measure of the product is whether a patient can walk them without anyone explaining anything.*

# The screens I expect to see

Nine screens. Fewer is a conversation, more is your choice.

## 01 SIGN UP AND LOG IN

- Registration, login, and the mascot lives here
- One account, patient or doctor

## 02 SEARCH AND RESULTS

- Search by specialty, doctor or clinic
- Filters: city or clinic, format, date
- Result cards showing the nearest free time

## 03 DOCTOR PROFILE

- Specialty, clinic, years of experience
- Services offered and the price of each
- The address they see patients at

## 04 THE CALENDAR

- A week at a time, days switchable
- Free times bookable, taken times visibly not
- A day with nothing free says so

## 05 CONFIRM THE BOOKING

- Doctor, service, place, date and time on one screen
- A reason for the visit, optional
- One confirm button, then a booking I can find again

## 06 MY APPOINTMENTS

- Upcoming and past, clearly apart
- Move or cancel from here
- Cancelled ones stay visible, marked

## 07 MY PROFILE

- Name, phone, email, date of birth
- Editable, and it sticks
- Language and theme remembered here

## 08 DOCTOR'S DAY

- Today and this week, ordered by time
- Patient, service, format, reason
- Mark completed or did not arrive

## 09 DOCTOR'S WORKING HOURS

- Which days, which hours, how long a visit takes
- Block a day off or a lunch break
- This is what produces the times patients see

— THE PRODUCT

# What has to work underneath

I cannot see any of this on a screen, and it is the entire reason the product is worth building.

## FREE TIMES ARE REAL

- The times a patient sees come from that doctor's actual working hours
- A visit has a length, and the day divides into it
- Days off, breaks and already-taken times are not offered
- Nothing in the past is offered, ever

## A TIME CAN BE TAKEN ONCE

- Two patients, two browsers, the same second, the same slot
- One of them gets it. The other gets a clear refusal, not an error page
- Not a check in the screen and not a check in your code alone
- We will walk this one together at the demo

## EVERYONE SEES ONLY THEIR OWN

- A patient sees their appointments and nobody else's
- A doctor sees their calendar and nobody else's
- True even if somebody edits the address bar
- A patient cannot act as a doctor by asking nicely

## AN APPOINTMENT HAS A LIFE

- Booked, moved, cancelled, completed, did not arrive
- Each move has an owner: who is allowed to make it
- Moving one frees the old time in the same breath
- Some moves cannot be undone, and you decide which

— NON-NEGOTIABLE

# Two I will not let go of

## ONE TIME, ONE PATIENT

The double booking from slide 2 has to be impossible, not unlikely. If the product survives everything except two people clicking at once, it has not solved the problem I described.

## EVERYONE SEES ONLY THEIR OWN

This is health information, so privacy is part of the product rather than a feature of it. Nobody should be able to reach somebody else's appointment by any route, and that is worth designing for early rather than patching later.

*Most of what is on these slides can be discussed if the weeks get tight. I would rather we protect these two.*

— SCOPE

# What I do *not* want you spending time on

× Medical records, test results, prescriptions, referrals

× Payments of any kind. Prices are shown, never charged

× Family profiles and booking on behalf of a child or a parent

× Reviews and ratings on doctors

× Email or SMS reminders and notifications

× Insurance, referral chains, state health registries

× A clinic administrator role. Patient and doctor is the whole cast

× Rooms, equipment, queues and waiting lists

× A native mobile app. The website working on a phone is enough

# Four things I want *because I want them*

Please treat these as part of the product rather than as polish for week 5. I will be looking for all four at the demo.

## LIGHT AND DARK

- Both themes designed properly, not one theme with inverted colours
- Light is the one I will look at first, and I want it soft and calm rather than stark
- First visit follows the setting on my computer
- A visible switch, and it remembers what I chose after a refresh
- No white flash on load when I am in dark
- Every screen, including errors and empty states, works in both

## A CUSTOM FONT

- One chosen typeface, not the browser default and not Arial
- Loaded with the product rather than pulled from someone else's server at page load
- Pick something that a person trusts with health information, and tell me why
- Latin and Cyrillic both, since the site is in two languages
- Text must not jump around while the font is loading

## TWO LANGUAGES

- English and Ukrainian, switchable from inside the site
- No text written directly into the screens, all of it in translation files
- Dates, times, prices and weekday names formatted per language
- Error and validation messages translated too, they are the ones everybody forgets
- My choice survives a refresh and a new login

## A MASCOT

- A character from medicine, on sign up and log in
- Generated with AI, in a soft 3D look, not bought from a stock library
- It has a name and a personality, and you tell me both
- It must not slow that screen down or fight the form for attention
- It has to look right in light and in dark, and on a phone

*The next slide is only about this one.*

# The mascot, *and who it is*

Booking a doctor makes people anxious. A friendly face on the first screen is the cheapest way to make the product feel like it is on their side. So I would like a character, not a decoration.

## WHO IT IS

- Give it a **name** and two or three sentences on who it is, and put them in the handover
- It comes from the world of medicine: a small doctor character, a stethoscope-wearing creature, a friendly bee in a white coat, your call
- A helper: calm, warm, competent

## HOW IT LOOKS

- Generated with AI, Claude does these well, and a soft 3D-render look is what I have in mind
- A light, friendly palette. Gentle colours that settle somebody down rather than clinical white and alarm red
- It should read well in light and in dark, and on a phone

## WHERE IT APPEARS

- Sign up and log in, always. That is the screen I want it for
- Nice to have: the empty state before a patient has any appointments
- It should never delay the form being usable

— THE ONE SLIDE WHERE I AM NOT YOUR CLIENT

# Your stack

Fastify

TypeScript

PostgreSQL

Drizzle

One repo

Both of you in it

Front end

Your own stack, your call

*You two keep TypeScript because you have the least free time of the four teams and a new language would cost you the lesson. Fastify gives you schema-validated routes, which a booking API wants, and plugins that encapsulate cleanly, which is how two people work in one back end without standing on each other. The back end is the point of this project. The front end is how I judge whether it works.*

— THE ONE SLIDE WHERE I AM NOT YOUR CLIENT

# Both of you, *not one of you*

You divide the work between yourselves, it is not something I want to decide for you. What I do ask is that either of you can answer a question about any part of it, so the project does not end with two people who each know half.

## HOW TO DIVIDE IT

- Split by surface, not by layer. Nobody owns "the back end" while the other owns "the front end"
- One natural cut: patient side (search, profiles, calendar, booking) against doctor side (working hours, the day view, auth and roles)
- Each surface owned end to end by one person: data, routes, screens
- The shared middle (the schema, the appointment rules) is designed together before either of you writes it
- Send me the split in writing at the end of week 1. One paragraph is enough

## CONCEPTS YOU BOTH OWN

- How free times are produced from working hours, and whether a slot is a row or a calculation
- What actually stops two bookings of one time, and where that guarantee lives
- What a transaction is, and which operations here need one
- How permissions are enforced per request and per object, not per button
- How times are stored and displayed, and what happens in another timezone
- The appointment states, who can move which, and what freeing a time means

# Your calls, *not mine*

I do not know the answers to these, and they are yours to decide rather than mine. You answer them in writing, in one spec from the two of you, and walk me through the answers in week 3.

- |                                                                                                         |                                                                                                                  |
|---------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| ? What a doctor's schedule is in your data, and how a week of bookable times comes out of it            | ? How the states move, who is allowed to move them, and what cannot be undone                                    |
| ? Whether a free time exists as a row before anybody books it, and what each answer costs you           | ? How you check that this patient may see this appointment, on every route, without repeating yourself ten times |
| ? What stops the same time being booked twice, from two browsers, in the same second                    | ? What you store when a time is written down, and what the patient sees if they open it from another country     |
| ? What happens when a doctor shortens their hours and appointments already sit in the part they removed | ? What your API looks like and what each of the nine screens genuinely needs from it                             |
| ? How moving an appointment takes the new time and releases the old one without ever losing both        | ? What you would do differently with three more months                                                           |

# What I will check, *one by one*

✓ I sign up, log out, log back in, and my appointments are still there

✓ Searching a specialty returns the doctors of that specialty, not everyone

✓ Two filters at once narrow the list instead of emptying it

✓ A doctor profile shows specialty, clinic, experience, services and prices

✓ The calendar shows that doctor's real hours, not a decorative grid

✓ A visit length divides the day, and lunch and days off are absent

✓ Nothing in the past can be booked, including earlier today

✓ A time I book disappears for everybody else straight away

✓ Two patients, two browsers, one time: one wins, the other is refused in plain language

✓ Moving my appointment gives the old time back to other patients in the same move

✓ Cancelling frees the time and the appointment stays visible, marked cancelled

✓ Upcoming and past are separated and in the right order

✓ A doctor logs in and sees their own day, in time order, with patient and service

✓ A doctor cannot see or touch another doctor's calendar

✓ A patient cannot open another patient's appointment, including by changing the address bar

✓ Editing my profile sticks after a logout

✓ Bad input gets a message a person understands, in the language they chose

✓ Light and dark both look designed, on every screen, with no white flash

✓ Switching language switches everything, dates and errors included, and it sticks

✓ The auth screen carries the mascot and still opens fast on a phone

# And how it has to be built

✓ The front end runs on your real back end. No mock data anywhere in the demo

✓ Everything lives in a real database and survives a restart

✓ Every request is checked on the server for who is asking and what they are asking about

✓ Hiding a button is not a permission. The check belongs on the server, where the request lands

✓ One time cannot be held twice even when both clicks land in the same instant, and you can show me why

✓ Times are stored one way and shown correctly, including from another country

✓ Seeded with enough real-looking data to judge it: several clinics, at least twenty doctors across specialties, real schedules, appointments in the past and the future

✓ The screens are designed, not assembled, and match the design you got approved

✓ It works on a phone screen, not only on your monitor

✓ Free times, booking and permissions have tests. Those three, at minimum

✓ A README that lets someone else run it from zero

✓ One spec, from both of you, matching what you actually built

✓ Either of you can explain the data model, the API and the booking rules out loud, alone, without notes

✓ Design, spec and code handed over together, in English

*Deploying it is optional. If you do deploy, that is the link I want at the demo rather than a screen share.*

— THE DEMO

# How the last day goes

This is the run I will ask for, in this order, live. Fifteen minutes, both of you present. Rehearse it beforehand so the day is about the product rather than about what is not working.

- |    |                                                                                                                                                        |    |                                                                                                                                                                       |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 01 | Open the site in a browser that has never seen it. It comes up in my system theme. Switch to the other one, switch the language to Ukrainian, and back | 08 | Now the important one. From a second browser, as a different patient, book that exact Tuesday 10:20. The site refuses in plain language. Then take 10:40 successfully |
| 02 | Sign up as a new patient on the auth screen, with the mascot on it. Tell me its name                                                                   | 09 | Move my appointment to Thursday at 11:00. Then show me, from the other browser, that Tuesday 10:20 is free again                                                      |
| 03 | Search for a cardiologist, filter to one city and to in-person, and open a doctor from the results                                                     | 10 | Log in as the doctor. Their day shows both patients in time order, with service and reason. Show me that this doctor cannot reach another doctor's calendar           |
| 04 | Show me the profile: clinic, experience, services and prices, and the address                                                                          | 11 | Still as the doctor, try to open a patient's appointment that is not theirs by changing the address bar. The server says no                                           |
| 05 | Open next week in their calendar. Show me a day with free times, a day that is full, and a day off                                                     | 12 | Back as the patient, cancel Thursday. It moves to past marked cancelled, and the time returns to the calendar                                                         |
| 06 | Book Tuesday at 10:20, add a reason for the visit, confirm, and land on a confirmation I can find again                                                | 13 | Open the whole thing on a phone and walk the same path far enough that I believe it                                                                                   |
| 07 | Open my appointments. It is under upcoming, with doctor, place and time                                                                                |    |                                                                                                                                                                       |

— WORKING TOGETHER

# What I expect from you

## EVERY WORKING DAY

- A short written update by the end of the day, in English
- Four lines: **done today, in progress, blocked, next.** That is the whole format
- **One update from the team, not two.** Agree it between you and send it
- Written the way you would write to a client, because I am one
- A day where nothing moved is still worth sending. Silence is the only version I mind

## TWICE A WEEK

- Fifteen minutes, screen shared, you show me the product
- Both of you on the call, and both of you talk
- Show me screens and behaviour, not code and not your terminal
- Show the unfinished thing. I would rather see it ugly in week 3 than perfect in week 6

## ONE VOICE

- Two people, one answer. Disagree with each other before you answer me, not during
- Questions about the product go to the team chat as they come up, not saved for the call
- If I am slow to answer, make the call yourself and tell me what you assumed
- I know you both have other work, so plan the six weeks around it early and tell me if the shape of it changes

— DURATION

# Six weeks, and what each one owes me

Started Wednesday 12 August 2026, finishing Wednesday 23 September 2026.  
Six weeks covering design, back end, front end and the demo.

| WEEK   | WHAT YOU ARE DOING                                                                            | WHAT I SEE AT THE END OF IT                                                     |
|--------|-----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| Week 1 | Read the case, agree the split between you, send me your questions, first pass at the screens | The split in writing, and a clickable flow of the main screens, rough is fine   |
| Week 2 | Design iterations after my review, including both themes and both languages                   | A design I have approved. Nothing gets built before this                        |
| Week 3 | Your back-end plan on paper: the schema, the API, the booking rules, the answers to slide 14  | One written spec, walked through out loud, both of you answering                |
| Week 4 | Back end alive on real data: doctors, schedules, free times, booking, roles                   | The rules working, double booking already refused, even with barely a front end |
| Week 5 | Front end on your real back end, patient side and doctor side end to end                      | I click through both surfaces myself, without you driving                       |
| Week 6 | The four personal wishes finished, tests, seed data, README, optional deploy                  | The demo on slide 17, rehearsed                                                 |

*The back end is the half of this I care about most, and weeks 3 and 4 are where it is won. A plain front end over a solid booking system is the better outcome of the two.*

— DONE

# What finished looks like

✓ A stranger books a doctor on it without me or a receptionist in the room

✓ The double booking from slide 2 is impossible, and you can show me why

✓ No patient can reach another patient's appointment by any route

✓ Moving and cancelling always leave the calendar telling the truth

✓ The doctor's morning screen matches what the patients booked

✓ The screens look designed, in both themes and both languages, mascot included

✓ The front end runs on your real back end, on real data

✓ Failures behave like a decision, not an accident

✓ Free times, booking and permissions have tests

✓ Either of you can explain the whole architecture, alone, without notes

✓ Design, spec and code handed over together

✓ The demo runs start to finish

*Two of you, one product. I should not be able to tell where one of you stopped and the other started.*

Ask the AI for everything. Ship nothing either of you cannot explain to me.